

GNU Tools that Help You Develop C/C++ Applications

This article explores how various GNU tools are used for the development of C/C++ applications. It also covers the anatomy of the generated intermediate files and the final outcome. These tools are also available for the Windows environment with support from the MingW runtime.



In the Linux environment, many GNU tools are installed by default when you select the C/C++ development category during installation, or use the package manager, post installation. Most of these tools are supported by packages like *gcc*, *glibc*, *binutils*, etc. You can use the package manager of a specific distribution to install missing packages or, on rare occasions, you can build these from the source code.

For Windows, similar tools are provided by the MingW suite, originally available from *mingw.org*; but this is limited to 32-bit versions and there have been no consistent updates recently. You may need to download multiple packages and merge them for recent versions. An installer utility is preferred to download all necessary packages in online mode. MingW generates code for the Windows runtime in the PE format, so UNIX/Linux-specific system calls can't be used with this.

You can get a TDM variant of *gcc*, which comes with the Code::Blocks IDE. Currently *gcc* v4.9.2 is bundled with Code::Blocks v16.01. With this you can also avail an elegant IDE with a *gcc* backend. Download *codeblocks-16.01mingw-nosetup.zip* from *codeblocks.org/downloads*, extract and then update the path to *codeblocks-16.01mingw-nosetup\MingW\bin* at the user level or system level. Now navigate to the directory holding the code in a command prompt, and run any command like *gcc*.

Alternatively, you can use the MingW-w64 Project, a fork of MingW, which supports 64-bit systems also and provides offline archives. You can download the archive from *sourceforge.net/projects/mingw-w64/files/*, then navigate to *Toolchains targeting Win64/Personal Builds/dongsheng-daily* and choose the desired version like 4.8, 4.9 or 5.x. Extract the archive and update the path to the extracted bin directory.

Optionally, the Cygwin Project also provides a complete UNIX-like environment in Windows with GNU tools and an abstraction layer for POSIX APIs. Any UNIX/Linux application can be ported on Cygwin.

A simple program and its analysis

To see various intermediate phases while building, let's look at the following simple example:

```
#simple.c
#include<stdio.h>
#define PI 22.0/7.0
int main()
{
    double area,r;      #a comment
    area=PI*r*r;
```